

Middleware

COMPLETE LEARNING GUIDE

What You'll Learn

Middleware is the mechanism nearly every backend framework uses to run shared logic — logging, authentication, parsing, error handling — on requests before (or after) they reach the final route handler, without repeating that logic in every route. Understanding middleware deeply is essential because it explains a huge share of 'why doesn't this work' bugs: a missing `next()` call, an authentication check registered after the routes it's supposed to protect, or an error thrown inside an async handler that never reaches the error handler. This guide walks through Express's middleware signature and pipeline model in depth, since it's the clearest and most explicit implementation of the pattern, and then connects it to the equivalent concepts in Django (middleware classes), Spring (interceptors/filters), and ASP.NET Core (the middleware pipeline), which all solve the same problem with different syntax.

1. What Middleware Actually Is: A Request/Response Pipeline

Conceptually, a web server processes each request through a pipeline of functions, each of which can inspect or modify the request/response, perform a side effect (logging, authentication), and then either pass control to the next function in the pipeline or stop the pipeline and send a response immediately. Middleware is simply the name given to those pipeline functions. This model is powerful because it lets cross-cutting concerns — things that apply to many or all routes, like parsing JSON bodies or checking authentication — be written once and composed onto whichever routes need them, instead of being duplicated inside every handler.

2. The Anatomy of Express Middleware

An Express middleware function has the signature `(req, res, next)` — it receives the request and response objects like a route handler does, plus a third function, `next`, which must be called to pass control to the next middleware or route in the chain. If a middleware never calls `next()` and never sends a response, the request hangs forever with no error and no timeout by default — one of the most common sources of a mysteriously 'frozen' Express endpoint. Middleware can be registered globally with `app.use()`, scoped to a path prefix with `app.use('/api', ...)`, or attached to a single route as an extra argument before the handler.

JAVASCRIPT

```
function requestLogger(req, res, next) {
  console.log(`${new Date().toISOString()} ${req.method} ${req.originalUrl}`);
  next(); // must call next() or the request hangs
}

app.use(requestLogger);           // runs on every request
app.use(express.json());          // built-in: parses JSON request bodies

// scoped to one route:
app.get("/api/users/:id", requireAuth, usersController.getUser);
```

3. Built-In and Third-Party Middleware You'll Use Constantly

Express ships with a small set of built-in middleware and relies on a large third-party ecosystem for everything else. `express.json()` and `express.urlencoded()` parse incoming request bodies into `req.body`, without which `req.body` would simply be undefined. `express.static()` serves files from a folder directly. Beyond the built-ins, `cors` handles Cross-Origin Resource Sharing headers, `helmet` sets a battery of security-related HTTP headers, and `morgan` logs HTTP requests in a configurable format — all near-universal in production Express apps.

- `express.json()` — parses `application/json` request bodies into `req.body`.
- `cors` — adds `Access-Control-Allow-*` headers so browsers permit cross-origin requests.
- `helmet` — sets security headers (`X-Content-Type-Options`, `Strict-Transport-Security`, etc.) with sane defaults.
- `morgan` — logs each request (method, path, status, response time) in one line.

4. Writing Custom Middleware for Cross-Cutting Concerns

Custom middleware is where application-specific cross-cutting logic lives: authentication checks, request-ID tagging for tracing, rate limiting, or attaching a tenant/organization context in a multi-tenant app. The key pattern is that middleware can attach data to `req` for downstream handlers to use — for example, an auth middleware verifies a token once and attaches the decoded user to `req.user`, so every route handler after it can simply read `req.user` without re-verifying anything.

JAVASCRIPT

```
const jwt = require("jsonwebtoken");

function requireAuth(req, res, next) {
  const token = req.cookies?.accessToken;
  if (!token) {
    return res.status(401).json({ error: "Not authenticated" });
  }
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    return res.status(401).json({ error: "Invalid or expired token" });
  }
}

module.exports = requireAuth;
```

5. Error-Handling Middleware: The Special Four-Argument Signature

Express identifies error-handling middleware purely by its arity — a function with exactly four parameters, (err, req, res, next), is treated as an error handler, and Express skips straight to it (instead of the normal pipeline) whenever `next(err)` is called with a truthy argument, or when a synchronous route handler throws. Error-handling middleware must be registered last, after all other `app.use()` and route calls, and there is typically only one per application, giving a single place to format every error response consistently.

JAVASCRIPT

```
// Must be registered AFTER all routes
app.use((err, req, res, next) => {
  console.error(err.stack);
  const status = err.statusCode || 500;
  res.status(status).json({
    error: status === 500 ? "Internal Server Error" : err.message,
  });
});
```

6. Ordering, Short-Circuiting, and the Cost of Getting It Wrong

Middleware order is not cosmetic — it's the entire behavior of the pipeline. Authentication middleware registered after the routes it's meant to protect protects nothing, because Express has already matched and run the unprotected route by the time the auth check would run. Similarly, `express.json()` must run before any route that reads `req.body`, or the body will be undefined. A middleware can 'short-circuit' the pipeline at any point by sending a response instead of calling `next()` — this is exactly how an auth failure stops a request from ever reaching its intended route handler.

- Global middleware (logging, body parsing, CORS) is typically registered first, before any routes.
- Auth middleware must be registered before the routes it protects — either globally or per-route.
- Error-handling middleware (4 arguments) must be registered last, after every route and other middleware.
- Calling `next()` a second time, or forgetting to call it at all, are the two most common middleware bugs.

7. The Same Pattern in Django, Spring, and ASP.NET Core

Django implements middleware as classes with `process_request/process_response`-style hooks, configured as an ordered list (`MIDDLEWARE`) in `settings.py` — the order in that list determines execution order exactly like Express's registration order. Spring uses `HandlerInterceptor` (for request/response lifecycle hooks around controller methods) and `Filter` (lower-level, servlet-spec middleware, often used for CORS/auth), both configured explicitly with an order. ASP.NET Core's middleware pipeline is built explicitly in `Program.cs` with `app.Use(...)` and `app.UseX()` calls, executing top-to-bottom on the way in and bottom-to-top on the way out — a nested, onion-like model that Express's flatter pipeline doesn't have, but the core idea (ordered, composable request processing) is identical across all four.

Common Pitfalls

- Forgetting to call `next()` inside a middleware function, causing the request to hang indefinitely with no error and no response.
- Registering authentication middleware after the routes it's supposed to protect, so it never actually runs before those routes.
- Reading `req.body` before `express.json()` (or an equivalent body-parsing middleware) has been registered, getting undefined instead of the parsed payload.
- Throwing inside an async middleware/route handler without a `try/catch` or a wrapper — Express does not automatically forward async rejections to error-handling middleware in versions before Express 5.
- Writing an error-handling middleware with the wrong number of parameters (accidentally 3 instead of 4), so Express treats it as regular middleware and never invokes it on errors.
- Calling `next()` and also sending a response in the same middleware, causing a 'Cannot set headers after they are sent' error.

Key Takeaways

- Middleware is a pipeline of (`req`, `res`, `next`) functions that run before a route handler, used for cross-cutting concerns like logging, parsing, and auth.

- A middleware that never calls `next()` and never responds hangs the request forever — this is the single most common middleware bug.
- Error-handling middleware is identified by its four-argument signature (`err, req, res, next`) and must be registered last.
- Order is behavior: body-parsing before routes that read `req.body`, auth before the routes it protects, error handlers after everything.
- Django middleware classes, Spring interceptors/filters, and ASP.NET Core's pipeline all implement the same ordered, composable request-processing pattern as Express.

Learn More

Express.js — Using Middleware

<https://expressjs.com/en/guide/using-middleware.html>

Express.js — Error Handling

<https://expressjs.com/en/guide/error-handling.html>

Microsoft Learn — ASP.NET Core Middleware

<https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware/>